

Projects in Robotics

Team Project

Full-Stack Mobile Manipulator

Team of 3–5 | Duration: 1 week

Objectives	Build, entirely in Python, a simulated mobile robot - kinematics, sensing, motion planning, probabilistic localization, and occupancy-grid mapping - and a simulated robot arm - forward/inverse kinematics, Jacobian-based control, dynamics, trajectory generation, and force/impedance control. Then integrate both into one end-to-end scenario.
Outcomes	1. A working Python simulation (Jupyter notebook or script(s)) demonstrating the full pipeline, 2. A short design-decisions writeup highlighting your design choices, as well as the capabilities and the limitations of your system, and 3. A short demo (5 minutes or less), showing your robot system.
Team	3–5 students. This project is entirely self-contained in Python — no ROS 2, Gazebo, or physical TurtleBot/manipulator access is required or expected, but if you would like to use ROS 2, Gazebo or physical TurtleBot, you are welcome to do so (in the bonus part).
Timeline	Step 1: kickoff, pick a scenario, assign tracks Steps 2 and 3: build the Mobile Base and Arm tracks in parallel Steps 4, 5 and 6: integration, debugging, and the contact-task demo Step 7: Prepare your writeup and record your demo.
Tools	Python 3.10+, NumPy, SciPy, SymPy, Matplotlib. The Python Control Library is optional, but welcome for Task 8.

Background

Every technical topic in this course has, so far, been exercised separately: kinematics and dynamics on a manipulator, motion planning and localization on a mobile robot, force control as its own standalone idea. **This project asks you to build all of it, together, in one system — entirely in simulation.** Because the whole thing runs in pure Python, there is no ROS 2 install, no Gazebo, and no hardware dependency: everything here is something you can build, test, and demo on a laptop.

The system you will build has two halves that meet at the end: a **mobile base** that navigates an unknown 2D environment, while localizing itself and building a map, and a **robot arm** that performs a contact task once the base arrives at its destination. The specific story around *why* the robot is doing this is up to you - please pick one of the three scenarios below.

Choose Your Scenario

Scenario A — Warehouse Restocking Robot

The robot navigates a warehouse aisle it has no pre-built map of. It localizes against a few shelf-end beacon landmarks, and reaches into a shelf slot to make contact with an item using a gentle, constant approach force - too much force “crushes” it, too little never registers contact.

Scenario B — Search-and-Rescue Inspection Robot

The robot explores an unknown, rubble-strewn area (obstacles are not known ahead of time) to find a target mechanism - a valve or lever - and operates it with a bounded, controlled torque, so a stuck or delicate mechanism isn't damaged.

Scenario C — Assistive Household Robot

The robot navigates a simulated room (furniture as obstacles) to reach a person or object, then hands over an item or presses a switch using admittance control — the arm must yield safely on unexpected contact, not push through.

All three scenarios use the exact same task list, timeline, and grading rubric below — they differ only in your environment layout and your choice of contact-force behavior for Task 10. Pick whichever motivates your team most.

Task 0 — Before You Start

Team & Scenario Registration — due end of day tomorrow, Tuesday, 8/11

Before any technical work begins: one team member must submit, via email to the instructor, your team's full member list and your chosen scenario (A, B, or C). This is due by the end of the day tomorrow, 8/11.

Task List

Mobile Base Track

#	Task	Covers
1	Kinematic motion model - implement a differential-drive (or unicycle) kinematic model: given wheel/velocity commands (v , ω), integrate pose (x , y , θ) over time.	Spatial representations & kinematics
2	Environment & sensor simulation - build the 2D grid-world environment (obstacles per your scenario) plus a synthetic sensor model: noisy odometry and range/bearing readings to landmarks (or simulated lidar rays).	Robot Sensing
3	Motion planning - implement a potential-field controller and/or a sampling-based planner (PRM or RRT) to compute a collision-free path from start to goal.	Motion Planning I & II
4	Probabilistic localization - implement a Kalman filter (EKF) or particle filter fusing noisy odometry with landmark/range measurements to estimate pose over time.	Probabilistic Localization
5	Occupancy grid mapping (SLAM-lite) - build a log-odds occupancy grid of the environment as the robot drives, using sensor readings and the estimated pose.	SLAM & Autonomous Navigation

Arm Track

#	Task	Covers
6	Forward & inverse kinematics - build a DH model for a 2–3 DOF planar arm; implement FK and numerical IK (Jacobian pseudoinverse) to reach a target.	FK/DH, Inverse Kinematics
7	Jacobian & velocity control - compute the manipulator Jacobian; use it for resolved-rate control and to check manipulability/singularities near the target.	Jacobians & Differential Motion
8	Dynamics & joint control - implement a basic dynamics model (inertia/gravity terms) and a PID or computed-torque controller tracking commanded joint trajectories.	Manipulator Dynamics & Control

9	Trajectory generation - generate a joint-space or Cartesian trajectory (polynomial or trapezoidal profile) from the current configuration to the target/contact pose.	Trajectory Generation
10	Force/impedance control for the contact task - implement hybrid position/force control or impedance/admittance control for your scenario's final contact task.	Force/Impedance Control

Integration

#	Task	Covers
11	End-to-end integration and demo - wire the mobile base (Tasks 1–5) and arm (Tasks 6–10) into one pipeline: navigate to the scenario's target, then execute the contact task. Produce a final animation/visualization for your demo, and record your demo. Finally, prepare a short writeup, explaining your team's design-decisions, and highlighting the capabilities and the limitations of your system.	Ties the full course together

Suggested Team Roles

This is a starting point, not a requirement — adjust to your team's strengths. For a team of 4:

- **Person A:** Tasks 1 & 3 (motion model + motion planning)
- **Person B:** Tasks 2, 4 & 5 (sensing + localization + mapping)
- **Person C:** Tasks 6, 7 & 8 (kinematics + Jacobian + dynamics/control)
- **Person D:** Tasks 9 & 10 (trajectory + force/impedance control)

Everyone contributes to Task 11 (integration) - this is deliberately the task most likely to surface problems in Tasks 1–10, so budget real time for it rather than treating it as a final afternoon's work. For a team of 3, merge Person A and B's tracks, or Person C and D's.

Suggested Timeline

Focus
Kickoff: pick a scenario, assign tracks, agree on shared interfaces (e.g., what a “pose” and a “map” look like as data structures, since both tracks will need to hand these to each other later on).
Mobile Base Track: Tasks 1–2, then 3–4 in parallel where possible.
Arm Track: Tasks 6–7, then 8.
Mobile Base: Task 5 (mapping). Arm: Tasks 9–10 (trajectory + force/impedance).
Task 11: integration. This is where the two tracks' outputs (a final pose, a map, a target location) actually have to connect - expect this to take longer than it looks.
Prepare your demo and your writeup.

Deliverables

All deliverables below are submitted via Canvas.

- **Code:** a Jupyter notebook (or a small set of Python scripts/modules) implementing all 11 tasks, runnable top-to-bottom without errors.
- **Demo:** a short (5-minute) live walkthrough showing the mobile base navigating, localizing, and mapping, followed by the arm's contact task.

- **Design-decision Writeup:** 2-3 pages covering: which scenario you picked and why; key design decisions per task (e.g., EKF vs. particle filter, potential field vs. RRT, hybrid vs. impedance control, and why); what worked, what didn't, and what you'd do with more time. Please also highlight the capabilities and the limitations of your system.

Using AI Tools on This Project

You're welcome to use AI tools (ChatGPT, Claude, Copilot, or similar) while working on this project — used well, they're a genuinely good way to learn faster, not just a shortcut. There's no live Q&A on this project, so the guidelines below are built around what you actually submit: your code and your writeup.

Always fine, no disclosure needed. Explaining a concept you are stuck on, debugging an error message, looking up syntax or library usage, or generating boilerplate (plotting code, argument parsing, etc.) — use AI for this as much as you'd like.

Fine, but disclose it and explain it in your writeup. AI-generated code for a specific, well-defined function or algorithm step (e.g., “implement RRT given this pseudocode”) is allowed — but only after your team has reviewed, tested, and understood it well enough to describe it in your own words. Say what you used it for in a sentence or two.

Not permitted. Asking AI to generate an entire task (or your team's overall integration/architecture) directly from the assignment description, or submitting AI output your team hasn't tested or can't explain.

You own whatever you submit. If it's wrong, buggy, or doesn't fit your scenario, that's your team's responsibility to catch — regardless of where it came from.

Tips for a Focused Build

1. Agree on interfaces before writing code. Decide early what a “pose,” a “map,” and a “target” look like as Python data structures (tuples? NumPy arrays? a small class?) — mismatched interfaces are the single most common source of integration pain later on.

2. Keep every component small on purpose. A 2-DOF arm, a modest grid world, and 3–5 landmarks are enough to demonstrate every idea correctly. Scaling up is a stretch goal, not a baseline requirement.

3. Test each track in isolation first. The mobile base should navigate and localize correctly against a fixed, hard-coded “arrival pose” before the arm track's output is wired in — and vice versa.

4. Budget real time for Task 11. Every team that has built something like this underestimates integration time — budget a large, explicit block of your remaining time for it, not just a final afternoon.

5. A working simple version beats a broken ambitious one. If you have to cut scope late, cut a stretch goal (a fancier planner, a bigger map), never the end-to-end connection between the two tracks — the integration is the point of this project.

6. Save your working versions as you go. Once a piece works, commit it (git, or even just a timestamped copy) before you touch it again — a broken “improvement” is much cheaper to back out of than to rebuild from memory.

Demo & Writeup Checklist

Before your demo, confirm you have:

- ☐ All 11 tasks implemented and individually testable
- ☐ The mobile base navigating, localizing, and mapping in one run
- ☐ The arm reaching and executing the scenario's contact task
- ☐ A working end-to-end run connecting both tracks (Task 11)
- ☐ A 2-3 page writeup covering your design decisions
- ☐ A recording of your 5-minute demo.

Grading Rubric

100 points total, identical across Scenarios A, B, and C — grade the pattern of what your team built, not the specific narrative. Task 0 (team and scenario registration) is a prerequisite and is not itself scored.

Category 3 (Integration) is weighted heavily on purpose: a team that nails every individual task but never connects them should score noticeably lower than one that connects a simpler pipeline end-to-end.

1. Mobile Base Track — 30 points

Task	Full credit requires	Pts
1. Kinematic model	A differential-drive or unicycle model correctly integrates (v, ω) commands into a pose (x, y, θ) trajectory over time, verified against at least one hand-checkable case (e.g., driving straight, or a pure rotation in place).	6
2. Sensing simulation	A 2D environment with obstacles matching the chosen scenario, plus a noisy odometry model and a noisy landmark/range (or simulated lidar) model - noise must be genuinely present and tunable, not zero.	6
3. Motion planning	A potential field and/or a sampling-based planner (PRM/RRT) computes a collision-free path from start to goal in the team's environment, demonstrated on at least one non-trivial obstacle layout.	6
4. Localization	A Kalman filter or particle filter fuses odometry and landmark/range measurements into a pose estimate that is visibly more accurate than raw odometry alone, shown via a plot comparing estimated vs. true vs. odometry-only pose.	6
5. Occupancy mapping	A log-odds occupancy grid is built up from sensor readings and the estimated pose, visibly distinguishing free, occupied, and unknown cells by the end of a run.	6

2. Arm Track — 30 points

Task	Full credit requires	Pts
6. FK/IK	A DH table for a 2–3 DOF planar arm; forward kinematics computed correctly (verified against at least one hand-computed configuration); numerical IK converges to a given reachable target via the Jacobian pseudoinverse.	6
7. Jacobian/velocity control	The manipulator Jacobian is computed and used for resolved-rate control; the team can identify and discuss at least one singular or near-singular configuration for their arm.	6
8. Dynamics/control	A basic dynamics model (at least gravity, ideally plus inertia) informs a PID or computed-torque controller that tracks a commanded joint trajectory with visibly bounded tracking error.	6
9. Trajectory generation	A joint-space or Cartesian trajectory (polynomial or trapezoidal) is generated from the arm's current configuration to the scenario's target/contact pose, with a plotted position/velocity profile.	6
10. Force/impedance control	Hybrid position/force control or impedance/admittance control is implemented for the scenario's contact task, demonstrating bounded contact force/torque rather than an unbounded position-control failure.	6

3. Integration (Task 11) — 20 points

Criterion	Full credit requires	Pts
End-to-end connection	The mobile base's final estimated pose and map genuinely determine where the arm executes its task - not two demos run independently and shown side by side.	10
Working full run	A single run (or a small number of tunable-seed runs) completes the full scenario: navigate, localize, map, arrive, then execute the contact task, without manual intervention partway through.	6
Visualization	A clear final animation or figure sequence shows the whole run - path taken, map built, and the arm's approach/contact - not just printed numbers.	4

4. Writeup — 15 points

Criterion	Full credit requires	Pts
Design decisions explained	Clearly explains the key choices made per task (e.g., EKF vs. particle filter, potential field vs. sampling-based planner, hybrid vs. impedance control) and why, not just what was implemented.	6
Honest results	States what worked and what didn't, with specifics (e.g., "localization drifted when landmarks were sparse near the goal"), not only positive framing.	4
Capabilities & limitations noted	Explicitly states what the system can and cannot do, matching the writeup's own required capabilities/limitations section.	2
AI disclosure & understanding	If the team used AI for any "yellow-tier" work (see the AI Tools policy), the writeup discloses it and explains it in the team's own words, demonstrating real understanding — not just a tool credit line. Teams with no such usage receive full credit here automatically.	3

5. Demo — 5 points

Criterion	Full credit requires	Pts
Demo runs live (or a clean recording)	The team runs their pipeline live within the time slot, and shows their system running in a recorded 5-minute video.	5

Total: 30 (Mobile Base) + 30 (Arm) + 20 (Integration) + 15 (Writeup) + 5 (Demo) = 100 points.

A note on scope: per this assignment's own tips, a 2-DOF arm, a modest grid world, and 3–5 landmarks are sufficient for full credit on every category above. We will not deduct points for small scope choices that are clearly deliberate and clearly explained in the writeup. We will, however, deduct points for missing functionality.

Optional Bonus — ROS 2, Gazebo & TurtleBot Challenge (Not Graded)

This is not part of the 100-point rubric, has no deadline tied to this sprint, and does not affect your grade in any way. It exists purely for teams who want to take their working Python logic further, for their own learning and portfolio value — attempt it only after your graded submission (Tasks 0–11) is solid.

The idea is porting, not redoing: you've already built working logic for motion, sensing, planning, localization, and mapping in Python. This challenge asks you to re-target that same logic at ROS 2's plumbing (nodes, topics, Nav2, Gazebo) instead of your own simulator — not to reimplement any of it from scratch.

Stage	What it asks
1 (easiest)	Wrap your Task 1/2 motion model and sensor simulation as a ROS 2 node, publishing real <code>nav_msgs/Odometry</code> and <code>sensor_msgs/LaserScan</code> topics — so it looks like a real robot's data stream to anything listening.
2	Keep your own localization (Task 4) and map (Task 5), but swap in Nav2's costmap and planner in place of your Task 3 planner, and compare the resulting paths.
3	Run the full scenario inside Gazebo with a TurtleBot model — the same navigate → arrive → contact-task sequence, now in a real physics simulator instead of your 2D one.

Stages build on each other, but you don't have to do them in order or attempt all three — pick whichever matches your team's remaining time and interest.

Timing: completely decoupled from this project's deadline. There's no due date for this at all — whenever your team has something worth showing, reach out to the instructor – we will be happy to check your progress, and provide feedback.